Big Data y Minería de Datos

Unidad 3. Aprendizaje computacional

U3.1. Fundamentos del aprendizaje computacional

UNIVERSIDAD DE
MURCIA

Unidad 3. Aprendizaje computacional
Estructura

UNIVERSIDAD DE MURCIA

- Fundamentos y definiciones
- Aprender de los ejemplos
- Fases del aprendizaje computacional



Big Data y Minería de Datos
Contenidos

UNIVERSIDAD DE MURCIA

Introducción a la minería de datos

Técnicas de inteligencia artificial

Reglas
Fundamentos lógicos
Sistemas basados en reglas
Reglas de asociación

Aprendizaje computacional
Fundamentos
Regresión y clasificación
No supervisado

Procesado del lenguaje natural
Fundamentos
Representación y tópicos
Sentimientos y NER

Introducción al big data

UNIVERSIDAD DE MURCIA



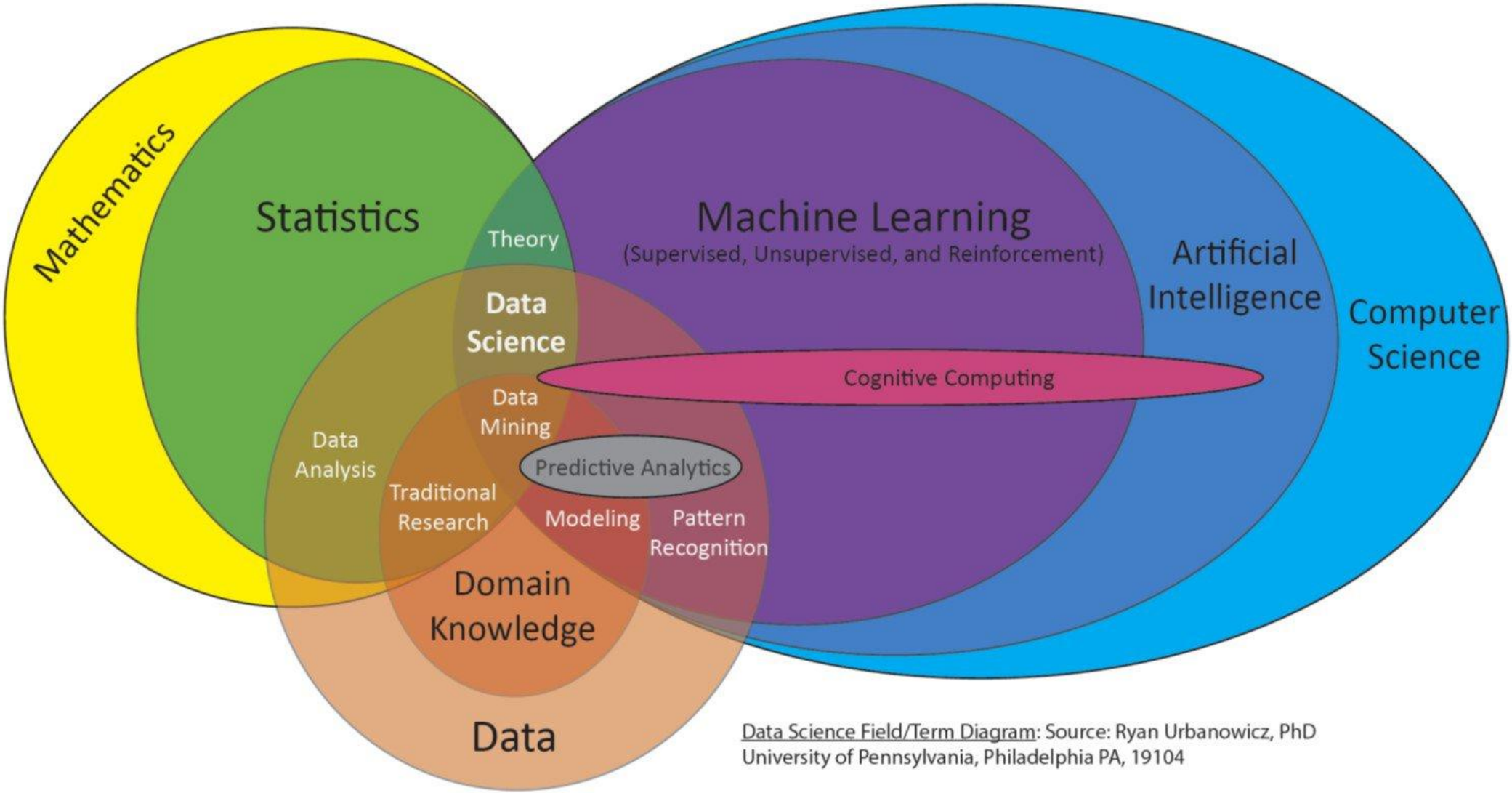
Unidad 3. Aprendizaje computacional

FUNDAMENTOS Y DEFINICIONES

- Aprender: Adquirir el conocimiento de algo por medio del estudio o de la experiencia (RAE)
- Computación: Que se adapta a ser tratado por computadoras
- Aprendizaje computacional (machine learning): desarrollo de técnicas o algortimos que permiten que las computadoras aprendan (Wikipedia)
  - También conocido como aprendizaje automático de máquinas o automatizado

Unidad 3. Aprendizaje computacional
Definiciones. Visión global

UNIVERSIDAD DE MURCIA



Data Science Field/Term Diagram: Source: Ryan Urbanowicz, PhD
University of Pennsylvania, Philadelphia PA, 19104

Unidad 3. Aprendizaje computacional
Objetivos de la IA: Actuar como un humano

UNIVERSIDAD DE MURCIA

- Quizás el objetivo más 'mainstream' de la IA
- El ordenador necesitaría las siguientes habilidades
  - Procesado del lenguaje natural
  - Representación del conocimiento
  - Razonamiento automatizado
  - Aprendizaje computacional
- Simulación física de una persona…
  - Visión por computador
  - Robótica
- **Las seis disciplinas principales de la IA**

Clasificación Clasica de los Enfoques de la IA

	RACIONAL	**HUMANO**
PENSAR	IA piensa de forma racional	IA piensa como un humano
ACTUAR	IA que actúa de forma racional	IA que actúa como un humano

- ¿Por qué debe aprender una máquina?
  - Inteligencia implica capacidad de aprendizaje
  - La experiencia es un elemento imprescindible
  - Un programa informático sin aprendizaje, seguirá devolviendo las mismas salidas
  - Un programa inteligente tras realizar cálculos incorrectos, modifica su comportamiento para rectificar (aprender)

- ¿Cuándo debería aprender una máquina?
  - Cuando en la tarea a resolver, no hay conocimiento experto, únicamente ejemplos almacenados en una base de datos
  - Cuando hay muchos datos para hacer descubrimiento
  - Cuando el conocimiento disponible para la tarea sobrepasa la capacidad de procesamiento humana (muchas características disponibles o no procesables por humanos)
  - Cuando los entornos son cambiantes y la máquina debe adaptarse automáticamente
  - Cuando se va generando nuevo conocimiento (por ej, vocabulario con nuevas tareas)

Introduction to ML, N.J. Nilsson, 1988

- ¿Cómo obtiene conocimiento una máquina?
  - Conocimiento por inducción: no conoce nada, y a través de ejemplos generaliza un patrón o regla (correcto o no)
  - Conocimiento a priori: la máquina parte de conocimiento, e incorpora nuevo adaptándolo
  - Conocimiento por deducción: se aplican unas reglas lógicas y se llega a una conclusión (nuevo conocimiento) que es verdadero si los hechos son ciertos)
  - Aprendizaje por transferencia (transfer learning): se desconoce como resolver tarea A, pero ha aprendido a resolver la tarea B y usa ese modelo para resolver A
  - Aprendizaje automático antagónico (adversarial learning): dos máquinas compiten para resolver una tarea y aprenden la una de la otra

UNIVERSIDAD DE MURCIA



Unidad 3. Aprendizaje computacional

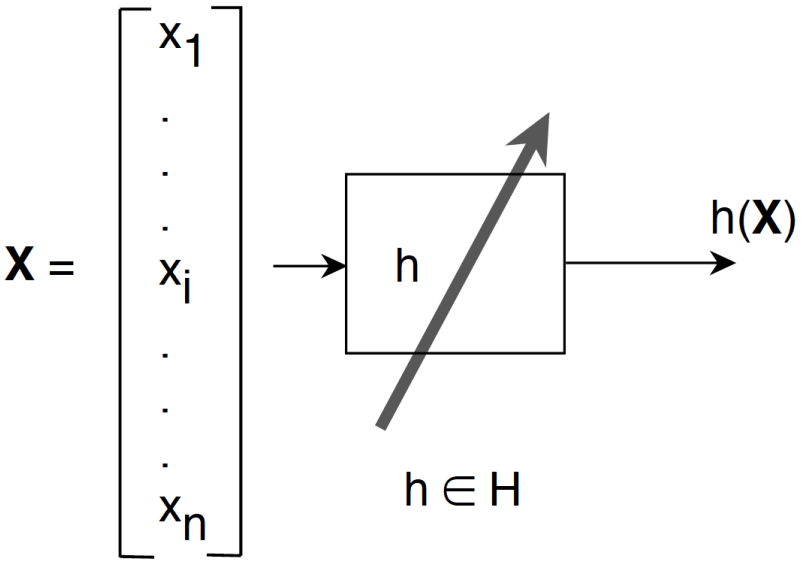
APRENDIENDO DE LOS EJEMPLOS

UNIVERSIDAD DE MURCIA

- Encontrar una función *f(X)* que es desconocida
- Buscamos una buena aproximación
- La aproximación es mediante una función *h(X)*, que tienen como entrada *X* con *n* componentes
- *h(x)* se selecciona en base a criterio del conjunto de *H(x)*, por ser buena aproximación
- El criterio de selección de *h(x)* es en base de un conjunto de entrenamiento de *m* ejemplares de X

Training Set:

$$\Xi = \{\mathbf{X}_1, \mathbf{X}_2, \ldots \mathbf{X}_i, \ldots, \mathbf{X}_m\}$$



- **Juego de datos (dataset)**: ejemplos proporcionados para aprender, son los casos de una base de datos
- **Modelo de aprendizaje computacional**: conocimiento aprendido en forma de estructura matemática y/o computacional
- **Método de aprendizaje computacional**: procedimiento o algoritmo a seguir para construir el modelo en base al juego de datos (de entrenamiento)
- **Parámetros del método**: conjunto de variables que ajustan el funcionamiento del método (durante el entrenamiento)

- **Juego de datos entrenamiento (train dataset):** juego de datos usado por el método para construir el modelo
- **Juego de datos de evaluación (test dataset)**: utilizado para evaluar el rendimiento del modelo
- **Predicción de un modelo**: una vez aprendido el modelo, es utilizado para obtener salida. Es la inferencia del agente
- **Rendimiento del modelo**: comprobar las prestaciones del modelo en base a diferentes criterios

- Se va a basar en cuatro factores principales:
- **Componente a mejorar**: aquello que queremos mejorar dentro de un contexto o problema
- **Conocimiento previo**: Información inicial que incorpora el modelo sobre el componente
- **Representación**: determina el algoritmo (método) a aplicar y que cosas se pueden modificar (parámetros) para mejorar el rendimiento
- **Retroalimentación (feedback)**: que le llega al modelo, para aprendizaje supervisado, no supervisado, semi-supervisado y por refuerzo

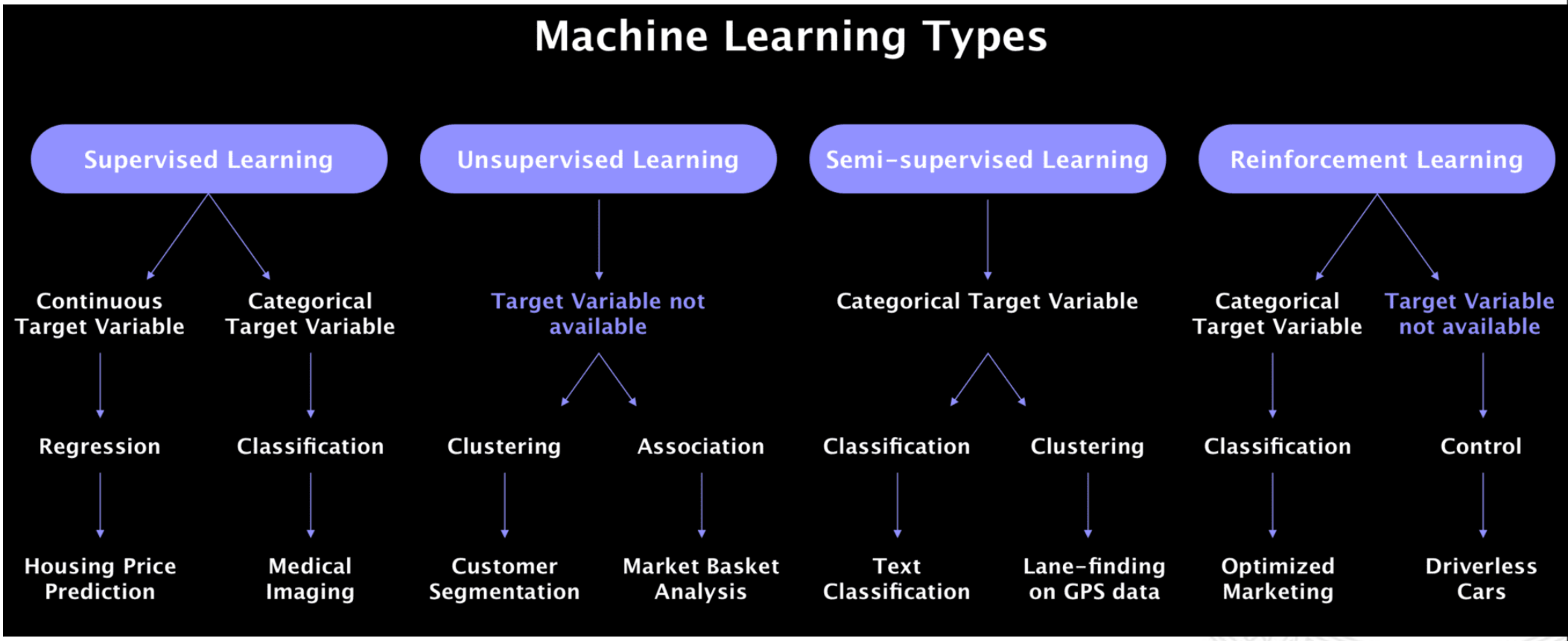
- Perspectiva clásica del agente y entorno, donde hay muchas componentes susceptibles de poder ser mejoradas
- Imaginemos el caso de un agente que sea un coche auto-conducido que aprende de la observación humana
  - El agente podría aprender una condición-acción de cuando frenar (c1)
  - Al ver muchas imágenes de cámaras con autobuses, aprende a identificarlos (c2)
  - Al probar acciones y observar resultados (por ejemplo, freno en agua), puede aprender consecuencias (c3)
  - En base a opiniones de pasajeros, puede aprender sobre la utilidad de sus acciones (c4)

- Nosotros vamos a asumir que empezamos con agentes que no conocen nada sobre el problema
- En otras ocasiones, podemos tener datos del problema que pueden ser introducidos de inicio al modelo
- También es posible introducir un modelo de un dominio en otro dominio
  - Puede aprender rápido con menos datos (por ej., en contextos de procesado de lenguaje natural o en la detección de imágenes)
  - Es un desafío y se aplica con más frecuencia en redes neuronales
- El diseñador debe elegir un marco de modelos que pueda llevar a un aprendizaje efectivo

- ¿En qué formato tenemos los datos para introducirlos en nuestro modelo?
- Nosotros trabajaremos sobre todo con una representación factorial en base a un vector de valores de los atributos
- También hay modelos con representaciones atómicas (búsqueda o modelos de Markov) o estructuradas (relaciones entre estados de un objeto LPO o compresión lenguaje natural)
- Ingeniería de características es el proceso de percibir entorno (capturar datos) e introducirlos al modelo
  - Por ej, de una base de datos al modelo
  - Por ej, de eventos en web a tiempo activo

- El tipo de retroalimentación que le llega al modelo es en sí una clasificación de los tipos de ML
- **Supervisado**: agente aprende a partir de conjunto de entrada (problema de inicio) - salida (solución deseada)
- **No supervisado**: el agente aprende en base a ejemplos donde no se conoce la solución (etiqueta)
- **Semi-supervisado**: usan una pequeña muestra de datos etiquetada (normalmente por humanos) y muchos no etiquetados (resto)
- **Por refuerzo**: el agente aprende a intentar resolver el problema, dado unos estados y acciones disponibles, en base a una función recompensa (premio/castigo)

- Según el **aprendiz**:
  - **Pasivo**: se le proporciona un juego de datos y parámetros para obtener un modelo de aprendizaje
    - El más común en el ML
  - **Activo**: de forma interactiva, identifica necesidades y solicita información concreta para la construcción del modelo
- Según la **salida**:
  - **Clasificación**: la salida pertenece a un conjunto finito de valores (tipo de animal)
  - **Regresión**: la salida pertenece a un conjunto infinitito (normalmente enteros o reales) (peso del animal)



https://litslink.com/blog/an-introduction-to-machine-learning-algorithms

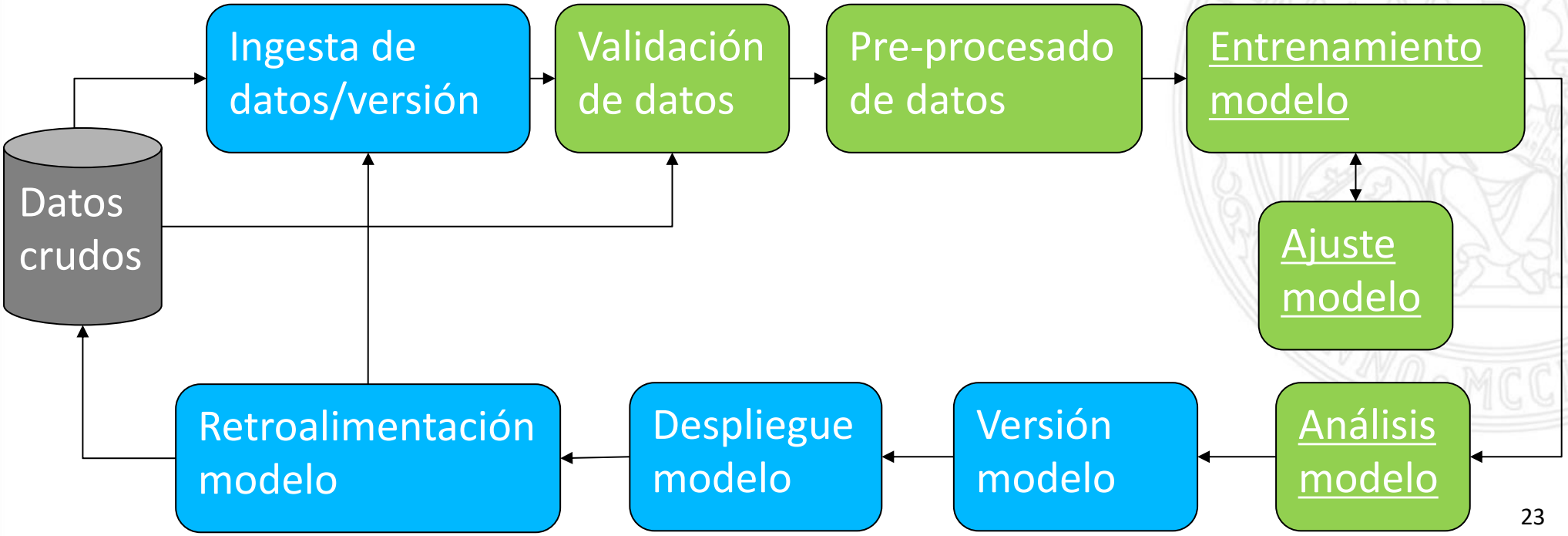
UNIVERSIDAD DE MURCIA

Unidad 3. Aprendizaje computacional

FASES DEL APRENDIZAJE COMPUTACIONAL

UNIVERSIDAD DE MURCIA

- Fases desde el punto de vista clásico offline (investigación o modelos retrospectivos) [verde]
- Fases desde el punto de vista de sistemas y despliegue de aplicaciones en producción [azul + verde]
- Llamamos pipeline (tubería) de aprendizaje computacional a las distintas fases/arquitectura (con el objetivo de automatizarlo)



- Es el comienzo de un pipeline de aprendizaje computacional
- Transformar los datos en formato en crudo a uno digerible por el sistema
- Esta etapa inicial no realiza la ingeniería de características
- Se debe mantener snapshot (foto los datos en un momento determinado) histórico de los datos, que se asocia a la versión del modelo entrenado (en una fase posterior) para garantizar la reproducibilidad

- Validar datos adecuados: generar estadísticas descriptivas (rango, categorías, datos anómalos o perdidos, etc)
- Automatización con test unitarios
- Comparación entre diferentes datasets
- Analizar la distribución entre clases para estudiar la necesidad de adaptar el muestreo de clases o evaluación
  - Subsampling/Oversampling, adaptar la función de pérdidas, obtener nuevos datos…

- Pre-procesados típicos generales (codificación one-hot, z-scores, normalización)
- Procesados específicos del dominio
  - Por ej., en texto, frecuencia de palabras, lematizar…
  - Por ej., en audio eliminar ruidos con filtro paso alto…
- Ingeniería de características (generar nuevas variables)
  - Calcular tiempo activo en web en base a eventos
  - Calcular perfiles de clientes con agrupamiento
- Eliminación de datos atípicos, si aplica
- Reducción de dimensionalidad
- Selección de variables
- Etapa frecuentemente más crítica que el modelo en sí (cada uno de los bloques podría ser su propia etapa)

- Paso principal de la construcción de un modelo
  - Entrena siguiendo método (algoritmo) elegido para minimizar el error
- El ajuste del modelo es el proceso en el que seleccionamos sus hiperparámetros
  - Se usa una malla de hiperparámetros y validación cruzada
- Puede ser que se haga antes del pipeline y luego se despliegue como parte del pipeline (por ej, si no se esperan cambios, si va a ser estático…)
- Computacionalmente muy pesado, en especial con determinados modelos y grandes juegos de datos
  - Técnicas de paralelización: datos, modelo o híbrido (o 3d)

- Una vez que se tiene una versión final del modelo se puede hacer un análisis en más profundidad
  - Métricas de calidad variadas
  - Análisis de errores por tipo y casos específicos
  - Análisis de características más útiles
- Evaluar si el modelo se comporta forma justa
  - Analizar la equidad del modelo, como se comporta el modelo con grupos de datos minoritarios (necesidad de dividir los datos en rebanadas en base a minorías)
  - Cambiar variables y analizar los cambios en las predicciones

- Este paso tiene como objetivo versionar el modelo, con el conjunto de hiperparámetros, datasets, arquitectura que han sido seleccionados
- El versionado semántico en ingeniería de software requiere cambiar el número mayor si es incompatible (o grandes cambios), o el número menor si no lo es
- Aquí también entra en juego el dataset (a veces influye más que la configuración)
  - ¿Mejor rendimiento implica nueva versión?

- Muchos modelos se implementan ad-hoc, y su actualización posterior es complicada
- Las versiones modernas permiten acceder a modelos a través de APIs (como REST), que incluso permiten varios modelos
- Hospedar varios modelos permite A/B testing (comparar dos versiones para ver cual tiene mejor rendimiento) y recibir retroalimentación
- Servidores modernos permiten actualizar modelos sin necesidad de re-desplegar la aplicación
  - Reduce el “tiempo de no disponibilidad” (downtime) de una aplicación

- Último paso, a veces olvidado, pero crítico para éxito
- Permite cerrar el bucle
- Capturar rendimiento del modelo en el entorno final
- Para actualizar el dataset, es posible que esto incluya al humano (human-in-the-loop) o puede ser automático

- En la actualidad aún no se considera un paso crítico, pero es inevitable que se convierta en tal
- Diferentes técnicas van emergiendo para hacer este proceso ML más privado
  - Privacidad diferencial: Funciones matemáticas que permiten compartir informaciones grupales, reteniendo información individual y calculando diferencias
  - Aprendizaje federado: Los datos no llegan a salir del dispositivo del usuario
  - Aprendizaje computacional encriptado: Datos encriptados y algoritmos o modelos adaptados a la encriptación